Studio Nx

Backend(FastAPI/Python) 37 — update_me()

2026-10-12

Backend(FastAPI/Python) 37 — `update_me()` (+ `MeUpdateRequest`)

항목	내용
파일	`studionx/backend/app/services/user_service.py`, `app/schemas/user.py`
작업	P1-08 회원 정보: 닉네임 변경 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	36 `to_me_response()` · 38 `save_profile_image()`

1. 하는 일

내 닉네임을 바꿉니다. 가입할 때와 같은 규칙(2~20자, 다른 사람과 겹치지 않음, 탈퇴 3일 규칙)을 지킵니다. 이메일은 바꿀 수 없습니다 (FR-106).

지금 닉네임 → 보낸 값	결과
`my_name` → (칸 없음), `null`, `"my_name"`	아무것도 하지 않습니다
`old_name` → `" New_Name "`	앞뒤 공백을 지운 `New_Name`으로 저장합니다
`Demo_Case` → `"demo_case"` (대소문자만 다름)	저장합니다. 내 닉네임이기 때문입니다
`my_name` → `"taken_name"` (다른 사람이 `Taken_Name`을 쓰는 중)	409 `NICKNAME_TAKEN` "This nickname is already in use."
`my_name` → `"gone_nick"` (어제 탈퇴한 회원의 닉네임)	409 `REJOIN_TOO_SOON` "This nickname can be used again after Oct 9, 2026 (UTC)."
`my_name` → `"free_again"` (4일 전에 탈퇴한 회원의 닉네임)	저장합니다. 탈퇴 회원 쪽 닉네임은 `deleted_user_0000000085`로 비켜 둡니다 (P1-04 `release_withdrawn`)
`" a "`, 21자	422입니다. 스키마에서 막힙니다 (아래 설명)

2. 언제 쓰나

쓰는 곳	
`PATCH /users/me` (41)	본문 `{"nickname": "…"}`를 받아 이 함수를 부르고, 36번으로 응답을 만듭니다
화면	userInfo.html의 닉네임 칸, Check 버튼, Save changes (P1-10)

템플릿에서 바꾼 것 2가지

	템플릿	바꾼 것	이유
스키마 `MeUpdateRequest.nickname`	`str`과 길이 2~20	가입과 같은 **`Nickname`** 타입	템플릿대로라면 `" a "`가 5자로 통과하고, 저장할 때 공백을 지우면 1자짜리 닉네임이 생깁니다. `Nickname`은 공백을 먼저 지운 뒤 길이를 잽니다
"지금 닉네임과 다르면 확인"	글자가 다르면 확인	**DB가 같다고 판단하면** 확인을 건너뜀	DB의 비교 규칙(utf8mb4_0900_ai_ci)은 대소문자를 구분하지 않습니다. 그래서 `Demo_Case` → `demo_case`를 확인하면 **내 닉네임**이 "사용 중"으로 나와 바꿀 수 없게 됩니다

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`select`, `IntegrityError`, `AppError`, `ErrorCode`, `check_nickname`을 이번에 추가했습니다.

스키마 (`schemas/user.py`)

```
from app.schemas.auth import Nickname

class MeUpdateRequest(BaseModel):
    nickname: Nickname | None = None
```

함수

```
def update_me(db: Session, user: User, data: MeUpdateRequest) -> User:
    nickname = data.model_dump(exclude_unset=True).get("nickname")
    if nickname is None or nickname == user.nickname:
        return user

    mine = db.scalar(
        select(User.user_id).where(User.user_id == user.user_id, User.nickname == nickname)
    )
    if mine is None:
        result = check_nickname(db, nickname, release=True)
        if result.reason == "REJOIN_TOO_SOON":
            when = result.available_at
            raise AppError(
                ErrorCode.REJOIN_TOO_SOON,
                409,
                f"This nickname can be used again after {when:%b} {when.day}, {when.year} (UTC).",
                details={"nickname": "REJOIN_TOO_SOON", "available_at": when.isoformat()},
            )
        if not result.available:
            raise AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.")

    user.nickname = nickname
    try:
        db.commit()
    except IntegrityError:
        # 확인과 저장 사이에 다른 사람이 같은 닉네임으로 먼저 바꾼 경우
        db.rollback()
        raise AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.") from None
```

```
    db.refresh(user)
    return user
```

문법 풀이

import 문법	뜻
`from app.services.auth_service import check_nickname`	P1-04(19)의 닉네임 확인 함수를 그대로 씁니다
`from sqlalchemy.exc import IntegrityError`	UNIQUE 같은 DB 규칙을 어겼을 때 나는 오류입니다

식	뜻
`Nickname | None = None`	닉네임 타입이거나 `None`이고, 보내지 않으면 `None`입니다
`data.model_dump(exclude_unset=True)`	**보낸 칸만** 사전으로 꺼냅니다. 보내지 않은 칸은 빠지므로, 나중에 칸이 늘어나도 보내지 않은 칸은 바뀌지 않습니다
`.get("nickname")`	칸이 없으면 `None`입니다
`select(User.user_id).where(A, B)`	`WHERE A AND B`입니다. 그 회원이면서, DB 규칙상 닉네임이 같은 행을 찾습니다
`db.scalar(…)`	첫 행의 첫 값을 가져오고, 없으면 `None`입니다
`f"{when:%b}"`	날짜를 `Oct`처럼 달 이름으로 씁니다
`raise … from None`	원래의 DB 오류(IntegrityError)를 오류 기록에 붙이지 않습니다. 예상한 상황이기 때문입니다

4. 흐름

1. 보낸 닉네임을 꺼냅니다. 없거나 지금 닉네임과 똑같으면 그대로 돌려줍니다.
2. DB가 "내 닉네임과 같다"고 판단하는지 확인합니다 (대소문자나 악센트만 다른 경우).
3. 같지 않다면 P1-04의 닉네임 확인을 거칩니다. 탈퇴한 지 3일이 안 된 닉네임이면 409 `REJOIN_TOO_SOON`, 사용 중이면 409 `NICKNAME_TAKEN`으로 응답합니다. (3일이 지난 탈퇴 닉네임은 이 안에서 비켜 둡니다.)
4. 저장합니다. 그사이 다른 사람이 먼저 가져갔다면(UNIQUE 위반) 되돌리고 409로 응답합니다.
5. DB에서 다시 읽어서 돌려줍니다.

5. 왜 이렇게

선택	이유
똑같으면 DB를 건드리지 않음	화면에서 닉네임을 바꾸지 않고 Save를 눌러도 불필요한 쓰기와 확인이 일어나지 않습니다
`null`도 "바꾸지 않음"으로 처리	닉네임은 비울 수 없습니다 (DB `NOT NULL`). 오류를 내는 것보다 무시하는 편이 안전합니다
탈퇴 3일 문구를 따로 만듦	P1-04의 `unavailable_error`는 "You can **sign up** with this nickname again after…"처럼 가입용 문구입니다. 이미 검토가 끝난 함수라 고치지 않고, 여기서 수정 화면에 맞는 문구를 만들었습니다
`release=True`	3일이 지난 탈퇴 닉네임은 가입할 때처럼 바로 쓸 수 있어야 합니다 (FR-108). 비켜 두는 변경은 같은 `commit`으로 함께 저장되고, 409로 끝나면 저장하지 않으므로 함께 취소됩니다
`IntegrityError`를 409로 바꿈	P1-04 가입과 같은 이유입니다. 확인과 저장 사이의 짧은 틈을 막습니다
마지막에 `refresh`	DB가 바꾼 값(`updated_at`)까지 받아서 36번 응답에 씁니다

6. 시험 (`tests/test_user_service.py`에 14개를 추가해서 모두 22개, 실제 MySQL로 실행)

시험	확인한 것
스키마	`" ab "`는 `"ab"`가 되고 20자는 통과합니다. `" a "`, 21자, 빈 문자열은 오류입니다
바꿀 것이 없음 (칸 없음, `null`, 같은 값)	같은 객체를 돌려주고, DB는 그대로입니다
새 닉네임 `" New_Name "`	DB에 `New_Name`이 저장됩니다
대소문자만 다름 (`Demo_Case` → `demo_case`)	저장됩니다
다른 사람의 닉네임 (대소문자만 다름)	409 `NICKNAME_TAKEN`이고, 내 닉네임은 그대로입니다
어제 탈퇴한 회원의 닉네임	409 `REJOIN_TOO_SOON`과 문구, `details`가 나오고, 내 닉네임은 그대로입니다
4일 전에 탈퇴한 회원의 닉네임	저장되고, 탈퇴 회원의 닉네임은 `deleted_user_{번호 10자리}`가 됩니다
동시에 바꾸기 (확인을 통과시켜 흉내 냄)	`commit`에서 UNIQUE 위반이 나서 409이고, 내 닉네임은 그대로입니다

전체 시험 256개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 DB에는 데모 계정과 사용자 계정만 남아 있습니다.